

# Tailwind CSS Tutorial: Advanced Level

*For people comfortable with theme customization, `@apply`, dark mode, and `group / peer`, who want to build a real design system and understand what's happening under the hood.*

---

## Table of Contents

1. [How Tailwind Actually Generates CSS](#)
2. [Writing Custom Plugins](#)
3. [Arbitrary Variants](#)
4. [CSS Variables as a Bridge to Dynamic Theming](#)
5. [Multi-Brand and Multi-Theme Architecture](#)
6. [Container Queries in Tailwind](#)
7. [The `safelist` and Dynamic Class Names](#)
8. [Tailwind v4: CSS-First Configuration](#)
9. [Performance at Scale](#)
10. [Accessibility Beyond the Defaults](#)
11. [Integrating with Component Frameworks](#)
12. [RTL and Internationalization](#)
13. [Practice Project: A Design-Token-Driven Component System](#)

## 14. [Where to Go Next](#)

---

# 1. How Tailwind Actually Generates CSS

Tailwind doesn't ship a giant pre-built stylesheet. Its **Just-in-Time (JIT) engine** scans the files listed in your `content` config, looking for anything that matches Tailwind's class-name syntax, and generates only the corresponding CSS on demand:

```
// tailwind.config.js
module.exports = {
  content: ['./src/**/*.html', './src/**/*.js', './src/**/*.jsx', './src/**/*.ts', './src/**/*.tsx'],
};
```

This is why a class name **must appear as a complete, static string** somewhere in your source — the scanner does a text search, not real code execution. Building class names dynamically like this will silently fail to generate CSS:

```
// This does NOT work — "text-" + color isn't a
// string Tailwind can find
const color = 'blue';
<div className={`text-${color}-500`}>
```

The fix is to write out full class names so the scanner can find them, even if it means a lookup object:

```
const colorClasses = {
  blue: 'text-blue-500',
  red: 'text-red-500',
};
```

```
};  
<div className={colorClasses[color]}>
```

Understanding this scanning behavior resolves the majority of "why isn't my class working" issues at this level.

---

## 2. Writing Custom Plugins

Beyond `@apply`, Tailwind's plugin API lets you generate entirely new utilities and components programmatically — useful when a pattern needs many systematic variations:

```
const plugin = require('tailwindcss/plugin');  
  
module.exports = {  
  plugins: [  
    plugin(function ({ addUtilities,  
      addComponents, theme, matchUtilities }) {  
      // Static utilities  
      addUtilities({  
        '.text-shadow': {  
          textShadow: '2px 2px 4px  
            rgba(0,0,0,0.3)',  
        },  
      });  
  
      // Reusable components  
      addComponents({  
        '.card': {  
          borderRadius:  
            theme('borderRadius.lg'),  
          padding: theme('spacing.6'),  
          backgroundColor:  
            theme('colors.white'),  
        },  
      });  
    },  
  ],  
};
```

```

    },
  });

  // Dynamic utilities generated from a
  // scale, e.g. .skew-15, .skew-30
  matchUtilities(
    { skew: (value) => ({ transform:
    `skewY(${value})` }) },
    { values: theme('skewValues') }
  );
},
],
};

```

`matchUtilities` is the key tool for generating a whole family of utilities (with arbitrary-value support built in) from one function, rather than hand-writing dozens of near-identical classes.

---

### 3. Arbitrary Variants

Just as arbitrary values let you drop in a one-off value, arbitrary variants let you apply a one-off selector or media condition without a plugin:

```

<div class="[&*&]:mb-4">
  <!-- applies mb-4 to all direct children -->
</div>

<div class="[@media(min-width:900px)]:flex">
  <!-- custom one-off breakpoint -->
</div>

```

```
<ul class="[&_li]:list-disc [&_li]:ml-4">
  <!-- targets descendant li elements -->
</ul>
```

Like arbitrary values, treat these as an escape hatch for genuinely one-off cases — a repeated pattern is a sign to promote it to a custom variant (Section 10 of the intermediate tutorial) or a plugin utility instead.

---

## 4. CSS Variables as a Bridge to Dynamic Theming

Tailwind's config is compiled at build time, so it can't respond to runtime state by itself. The standard pattern for genuinely dynamic theming (user-selected accent colors, white-labeling) is to point Tailwind's theme at CSS variables, then update the variables at runtime:

```
// tailwind.config.js
module.exports = {
  theme: {
    extend: {
      colors: {
        brand: 'rgb(var(--color-brand) / <alpha-value>)',
      },
    },
  },
};
```

```
:root {
  --color-brand: 14 165 233; /* space-separated
  RGB so opacity utilities still work */
}
[data-theme="forest"] {
  --color-brand: 22 101 52;
}
```

```
<button class="bg-brand/80 text-white">
  <!-- opacity utility (/80) still works because
  of <alpha-value> -->
</button>
```

This gets you the best of both: Tailwind's utility ergonomics, plus a color that can change at runtime via a single variable swap (e.g. a theme picker in a settings panel).

---

## 5. Multi-Brand and Multi-Theme Architecture

Combining Sections 1 and 4, a real multi-brand product typically:

1. Defines all design tokens as CSS variables scoped under a `[data-brand="x"]` attribute on `<html>` or a wrapping element.
2. Maps every Tailwind color/spacing/radius token to those variables in `tailwind.config.js`, never hardcoding brand-specific values directly into components.

3. Switches brands by changing a single attribute, with no rebuild required.

This keeps components brand-agnostic — a `<Button>` component never needs to know which brand it's rendering inside.

---

## 6. Container Queries in Tailwind

The official container queries plugin (built into Tailwind's core as of recent versions) lets components respond to their container's width rather than the viewport:

```
<div class="@container">
  <div class="flex flex-col @md:flex-row">
    <!-- switches to row layout once the
    CONTAINER (not viewport) hits @md width -->
  </div>
</div>
```

This is the single biggest unlock for truly reusable components — a card component can render correctly whether it's placed in a wide main area or a narrow sidebar, because its layout responds to its own box rather than the whole page.

---

## 7. The `safelist` and Dynamic Class Names

Sometimes class names genuinely can't be static — for example, colors coming from a CMS or API response. Since

the JIT scanner can't see runtime values, use `safelist` to force-generate specific classes regardless of whether they appear as plain text in your source:

```
// tailwind.config.js
module.exports = {
  safelist: [
    'bg-red-500',
    'bg-green-500',
    'bg-blue-500',
    {
      pattern: /bg-(red|green|blue)-
(100|300|500|700)/,
    },
  ],
};
```

Use `safelist` sparingly and deliberately — it works against JIT's main benefit (shipping only the CSS you use), so it should cover a known, bounded set of values, not act as a blanket fallback.

---

## 8. Tailwind v4: CSS-First Configuration

Recent Tailwind versions moved toward configuring the theme directly in CSS rather than a JavaScript config file, using the `@theme` directive:

```
@import "tailwindcss";

@theme {
  --color-brand: oklch(60% 0.15 250);
}
```



```
--spacing-18: 4.5rem;  
--font-display: "Poppins", sans-serif;  
}
```

This removes the separate `tailwind.config.js` for theme values, keeping tokens declared in plain CSS (which also plays well with the native CSS custom properties covered in Section 4). Plugin registration and content scanning configuration still exist but are increasingly expressed via CSS as well. If working across projects on different Tailwind versions, checking which configuration style a given codebase uses is worth doing before making theme changes.

---

## 9. Performance at Scale

- **Keep content globs precise.** An overly broad glob ( `./**/*` ) slows down the scanner and risks accidentally matching files (like `node_modules` ) that shouldn't be scanned.
  - **Avoid generating truly unbounded arbitrary values** (e.g., pulling a raw hex code from user input straight into a class) — prefer a fixed palette plus CSS variables (Section 4) so the generated CSS stays bounded and cacheable.
  - **Purge is automatic under JIT** — there's no separate "purge step" to configure in modern Tailwind; unused utilities simply never get generated in the first place.
-

## 10. Accessibility Beyond the Defaults

Utilities make it easy to *ship* accessible patterns, but they don't enforce them — a few habits worth building in:

```
<!-- Focus states should never be silently
removed -->
<button class="focus-visible:outline-none focus-
visible:ring-2 focus-visible:ring-brand">
  Accessible focus state, not just removed
</button>

<!-- Respect reduced motion -->
<div class="motion-safe:animate-bounce motion-
reduce:animate-none">
</div>
```

`motion-safe:` and `motion-reduce:` variants map directly to the `prefers-reduced-motion` media feature — use them on anything with a looping or attention-grabbing animation rather than assuming it's harmless.

---

## 11. Integrating with Component Frameworks

In React/Vue/Svelte codebases, the main architectural decision is **where utility strings live**:

```
// Simple: inline utilities directly on JSX
elements
function Card({ children }) {
  return <div className="rounded-lg shadow-md p-
```

```

6 bg-white">{children}</div>
}

// Variant-driven: use a library like `class-
variance-authority` (cva)
// to manage multiple style variants cleanly
const button = cva('rounded-md font-semibold
transition', {
  variants: {
    intent: {
      primary: 'bg-brand text-white hover:bg-
brand-dark',
      secondary: 'bg-gray-100 text-gray-900
hover:bg-gray-200',
    },
    size: {
      sm: 'px-3 py-1.5 text-sm',
      lg: 'px-6 py-3 text-lg',
    },
  },
});

```

`cva` (or similar variant utilities) becomes worthwhile once a component has more than two or three style variants – past that point, conditional string concatenation gets hard to read and maintain.

---

## 12. RTL and Internationalization

Tailwind's logical property utilities mirror the native CSS logical properties covered in the advanced CSS3 tutorial:

```

<div class="ms-4 me-2 ps-6">
  <!-- margin-inline-start, margin-inline-end,

```

```
padding-inline-start -->
</div>
```

Using `ms-*` / `me-*` (start/end) instead of `ml-*` / `mr-*` (left/right) means the layout automatically flips correctly when the page direction switches to RTL (e.g. `dir="rtl"` for Arabic or Hebrew), with zero additional CSS.

---

## 13. Practice Project

Build a design-token-driven component system:

1. Define brand colors, spacing, and type scale using CSS variables under `[data-brand]` and `[data-theme]` attributes, mapped into Tailwind's theme (Sections 4–5).
  2. Write a custom plugin that adds a `.card` component and a `matchUtilities`-based `skew-*` utility.
  3. Build a `<Button>` component (in your framework of choice) using `cva` with `intent` and `size` variants.
  4. Make a card component responsive to its container width using `@container` and `@md`: rather than a viewport breakpoint.
  5. Audit the whole system for `focus-visible`: states and `motion-reduce`: variants on anything animated.
- 

## 14. Where to Go Next

- **Tailwind's own plugin source code**  
(tailwindcss/tailwindcss on GitHub) — reading the official plugins is one of the fastest ways to learn advanced plugin-authoring patterns
- **Design systems in production** (e.g. shadcn/ui's source) to see cva-style variant architecture applied at scale
- **PostCSS** — Tailwind is itself a PostCSS plugin; understanding the broader PostCSS pipeline helps when integrating with other CSS tooling
- **Style Dictionary or Tokens Studio** for teams syncing design tokens between Figma and a Tailwind config/CSS variables

---

*Tip: at the advanced level, most of the real work has moved from "which utility class do I use" to "how do I structure tokens, variants, and plugins so the whole team stays consistent without fighting the system."*